i need to write code for detecting nostalgia from bangla text . text are collected from youtube. the decision will be nostalgic/not nostalgic label.

tell me indept about it. lets r&D

Thought for 6m 43s

Below is a practical, "research-to-code" roadmap for **detecting nostalgia in Bangla (Bengali) YouTube comments** with a **binary label: nostalgic / not nostalgic**. I'll cover: problem framing, annotation, preprocessing (Bangla + Banglish), modeling options (baselines → Transformers), evaluation, error analysis, and then give you **working code templates** you can adapt.

---

1) What exactly is "nostalgia" in Bangla text?

In text classification, your biggest win is having a crisp definition.

Nostalgia = a sentimental longing / warm-sad feeling about the past.
It's usually a *blend* of: memory + emotion + past-time reference.

Common Bangla nostalgia signals (examples)

These are *signals*, not rules:

- **Memory recall verbs/phrases**
  "মনে পড়ে", "মনে পড়ে গেল", "স্মৃতি", "মনে আছে", "ভুলতে পারি না"
- **Longing / missing**
  "মিস করি", "খুব মিস করি", "ফিরে যেতে চাই", "আবার যদি..."
- **Past-time anchors**
  "আগের দিন", "একসময়", "তখনকার", "সেই সময়", "পুরনো দিন"
- **Childhood/school life**
  "শৈশব", "ছোটবেলা", "স্কুল জীবন", "কলেজের দিন"
- **Old media triggers**
  "পুরনো গান", "সেই নাটক", "ছোটবেলায় এটা দেখতাম"

- **Interjections + emotion**
  "আহা!", "ইশ!", "কি দিন ছিল!", plus 😥 ❤️ 🥺

Non-nostalgic "past reference" (important!)

Many comments mention the past but are **not** nostalgic:

- Pure factual past: "ভিডিওটা ২০১৮ সালের"
- Complaint/comparison without sentimental longing: "আগে দাম কম ছিল"
- News/politics/history discussion without personal longing

So your classifier is really learning:

> **past reference + affect (sentimental longing)** → nostalgic
> otherwise → not nostalgic

---

2) Data strategy: YouTube comments are noisy (plan for it)

Typical YouTube Bangla comment issues

- Emojis, repeated letters: "ভাাালো লাগলো 😭 😭 "
- Spelling variance + dialect
- **Code-mixed Banglish**: "onek nostalgic lagse bro"
- Romanized Bangla: "amar chotobelar din mone pore"
- Spam, promotional text, hashtags

This changes preprocessing + model choice.

---

3) Labeling (the real "R&D core")

You can't model nostalgia well without **strong annotation guidelines**.

Recommended labeling scheme (still ends as binary)

During annotation, use **3 labels**:

1. **NOSTALGIC**
2. **NOT_NOSTALGIC**
3. **UNCERTAIN / CONTEXT-NEEDED**

Then later you can either:

- drop "uncertain", or
- merge into NOT_NOSTALGIC, or

- keep for analysis

This reduces “forced wrong labels” early.

A simple guideline you can give annotators

Label **NOSTALGIC** if the comment contains at least one of:

- personal memory recall (“মনে পড়ে…”, “ছোটবেলায়…”)
- longing/missing (“মিস করি”, “ফিরে যেতে চাই”)
- emotional warmth/sadness about past (“কি দিন ছিল” with sentiment)

Label **NOT_NOSTALGIC** otherwise.

Quality control

- Use **2 annotators** + adjudication for disagreements
- Track **Cohen’s Kappa** (agreement)
- Keep a “confusing examples” doc and update the guideline

---

4) Preprocessing that actually helps Bangla nostalgia detection

4.1 Normalize Bangla text (highly recommended)

For BanglaBERT-family models, BUET NLP notes that their models were pretrained with a **specific normalization pipeline** and recommend using it for best results. Hugging Face +2

You can use `csebuetnlp-normalizer`:

bash Copy code

```bash
pip install csebuetnlp-normalizer
```

python Copy code

```python
from normalizer import normalize
txt = normalize(txt)
```

(That `from normalizer import normalize` usage is shown in the package docs. PyPI)

4.2 Decide what to do with emojis

Don't blindly remove emojis. For nostalgia, emojis often carry the emotion.

Good options:

- Keep them as-is (Transformers can learn patterns)
- Or replace emojis with a token like `<EMOJI>` (the normalizer can do emoji replacement too). PyPI

4.3 Handle Romanized Bangla / Banglish (very common on YouTube)

You have 3 strategies:

Strategy A (simple): keep as-is and use a multilingual model (XLM-R, MuRIL).
Strategy B (better for Bangla-only models): transliterate Roman → Bangla first.
Strategy C: train separate models per script and ensemble.

For transliteration, AI4Bharat provides a ready engine (`ai4bharat-transliteration`) with example usage: PyPI

bash Copy code

```bash
pip install ai4bharat-transliteration
```

python Copy code

```python
from ai4bharat.transliteration import XlitEngine

e = XlitEngine("bn", beam_width=10, rescore=True)   # bn = Bangla
out = e.translit_word("chotobelar", topk=5)
print(out)  # returns candidates
```

That API pattern is shown in their PyPI usage section. PyPI

---

5) Modeling options (baseline → strong Transformers)

5.1 Strong baseline (do this first)

A surprisingly competitive baseline for short noisy comments:

- **TF-IDF character n-grams (3–5)**
- **Linear model** (LogReg or Linear SVM)

- `class_weight="balanced"` if nostalgic is rarer

Why char n-grams?

- Handles misspellings + elongated forms + mixed script fragments better than word-only features.

5.2 Transformers: which model should you try?

Pick based on your data:

If most comments are Bangla script
BanglaBERT is a strong Bangla-focused pretrained model (ELECTRA discriminator). Hugging Face +1

They also mention smaller variants that can fine-tune with lower VRAM. GitHub +1

- `csebuetnlp/banglabert` or `csebuetnlp/banglabert_large` Hugging Face +1

If lots of Banglish / code-mixed Bangla+English
BanglishBERT is pretrained on Bengali+English corpora and is meant for bilingual/code-mixed settings. Hugging Face +1

- `csebuetnlp/banglishbert` Hugging Face +1

If you have Romanized Bangla frequently
MuRIL was designed for Indian languages and includes transliterated counterparts in pretraining; it's often a good fit for noisy transliteration/code-mix. arXiv +1

- `google/muril-base-cased` Hugging Face

General multilingual fallback
XLM-R is a strong multilingual baseline across 100 languages. arXiv +1

---

6) Evaluation: don't get fooled by accuracy

Nostalgic comments are often a minority. Track:

- **Precision / Recall / F1 for nostalgic class (label=1)**
- Confusion matrix
- Calibration / threshold tuning (0.5 isn't always optimal)

Also do "**channel split" evaluation**:

- Train on comments from some channels/videos
- Test on entirely different channels/videos
  This reveals whether you learned nostalgia or just channel-specific phrases.

7) Code: End-to-end baseline (fast + strong)

Assume you have a CSV: `data.csv` with columns:

- `text`
- `label` (0 = not nostalgic, 1 = nostalgic)

python Copy code

```python
import re
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

URL_RE = re.compile(r"https?://\S+|www\.\S+")
WS_RE  = re.compile(r"\s+")

def clean_text(s: str) -> str:
    s = str(s).strip()
    s = URL_RE.sub(" ", s)
    s = WS_RE.sub(" ", s)
    return s

df = pd.read_csv("data.csv")
df["text"] = df["text"].map(clean_text)

X_train, X_test, y_train, y_test = train_test_split(
    df["text"], df["label"],
    test_size=0.2,
    random_state=42,
    stratify=df["label"],
)

model = Pipeline([
    ("tfidf", TfidfVectorizer(
        analyzer="char",
        ngram_range=(3, 5),
        min_df=2,
    )),
    ("clf", LogisticRegression(
```

```
        max_iter=2000,
        class_weight="balanced"
    )),
])

model.fit(X_train, y_train)
pred = model.predict(X_test)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, digits=4))
```

If this baseline is weak, it usually means:

- labels are inconsistent, or
- nostalgia is context-dependent in your dataset, or
- too much romanized/codemixed text without handling it

---

8) Code: Transformer fine-tuning (BanglaBERT / BanglishBERT / MuRIL)

Install

bash Copy code

```bash
pip install transformers datasets evaluate accelerate torch scikit-learn csebuetn
```

Training script

python Copy code

```python
import numpy as np
import pandas as pd
from datasets import Dataset
import evaluate
from sklearn.utils.class_weight import compute_class_weight

import torch
from torch import nn
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
```

```
    Trainer,
)
from normalizer import normalize  # csebuetnlp-normalizer

MODEL_NAME = "csebuetnlp/banglabert"   # try also: csebuetnlp/banglishbert, googl
MAX_LEN = 128

df = pd.read_csv("data.csv")
df["text"] = df["text"].astype(str).map(lambda x: normalize(x))
df["label"] = df["label"].astype(int)

dataset = Dataset.from_pandas(df)
splits = dataset.train_test_split(test_size=0.2, seed=42, stratify_by_column="lab

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tok(batch):
    return tokenizer(
        batch["text"],
        truncation=True,
        padding="max_length",
        max_length=MAX_LEN
    )

splits = splits.map(tok, batched=True)
splits = splits.rename_column("label", "labels")
splits.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"]

model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME, num_labels

# --- class weights (important if imbalanced) ---
classes = np.array([0, 1])
weights = compute_class_weight(class_weight="balanced", classes=classes, y=df["la
class_weights = torch.tensor(weights, dtype=torch.float)

class WeightedTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        logits = outputs.logits
        loss_fct = nn.CrossEntropyLoss(weight=class_weights.to(logits.device))
        loss = loss_fct(logits.view(-1, 2), labels.view(-1))
        return (loss, outputs) if return_outputs else loss
```

```
acc = evaluate.load("accuracy")
f1  = evaluate.load("f1")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {
        "accuracy": acc.compute(predictions=preds, references=labels)["accuracy"]
        "f1_pos":   f1.compute(predictions=preds, references=labels, average="bin
        "f1_macro": f1.compute(predictions=preds, references=labels, average="mac
    }

args = TrainingArguments(
    output_dir="models/nostalgia_bn",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=3,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="f1_pos",
    greater_is_better=True,
    logging_steps=50,
    fp16=torch.cuda.is_available(),
    seed=42,
)

trainer = WeightedTrainer(
    model=model,
    args=args,
    train_dataset=splits["train"],
    eval_dataset=splits["test"],
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)

trainer.train()
print(trainer.evaluate())
```

```
trainer.save_model("models/nostalgia_bn")
tokenizer.save_pretrained("models/nostalgia_bn")
```

Notes on model choice (practical)

- If your comments are heavy **Banglish**, start with `csebuetnlp/banglishbert` . Hugging Face +1
- If your comments contain lots of **romanized Bangla**, try `google/muril-base-cased` or transliterate first. arXiv +2
- If mostly Bangla script, `csebuetnlp/banglabert` is a strong default. Hugging Face +1

---

9) Inference code (predict nostalgia on new YouTube comments)

python Copy code

```python
import numpy as np
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from normalizer import normalize

MODEL_DIR = "models/nostalgia_bn"

tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_DIR)
model.eval()

@torch.no_grad()
def predict(texts, threshold=0.5):
    if isinstance(texts, str):
        texts = [texts]
    texts = [normalize(t) for t in texts]

    batch = tokenizer(texts, return_tensors="pt", truncation=True, padding=True,
    logits = model(**batch).logits
    probs = torch.softmax(logits, dim=-1)[:, 1].cpu().numpy()  # prob of label=1
    labels = (probs >= threshold).astype(int)
    return list(zip(labels.tolist(), probs.tolist()))

print(predict("ছোটবেলার দিনগুলো খুব মনে পড়ে 😢"))
print(predict("ভাই ভিডিওটা দারুণ হয়েছে"))
```

10) R&D experiments that usually move the needle

Experiment 1: Add “context” text

Often nostalgia depends on what the video is (old song/drama).
Concatenate inputs:

```
[comment] [SEP] [video_title] [SEP] [channel_name(optional)]
```

This is a big accuracy boost in many social-media emotion tasks.

Experiment 2: Domain-adaptive pretraining (DAPT)

If you have *lots of unlabeled YouTube comments*, continue MLM pretraining on them (even 50k–500k comments helps), then fine-tune for nostalgia.

Experiment 3: Weak supervision + active learning

- Start with rules to generate weak labels (keywords like “মনে পড়ে”, “ছোটবেলা”, “পুরনো দিন”, “মিস করি”, “nostalgic”)
- Train a model
- Then label the “most uncertain” samples (prob ~0.5) to improve fast

Experiment 4: Script-aware pipeline

- If Bangla script ratio high → BanglaBERT
- Else if romanized ratio high → MuRIL or transliterate then BanglaBERT
- Ensemble the probabilities

---

11) Useful public Bangla/YouTube data (for extra unlabeled/domain data)

Even if they’re not nostalgia-labeled, they’re useful for **domain pretraining** or text normalization testing:

- A Bengali YouTube news/public opinion comments dataset (2017–2023) exists with hundreds of thousands of comments. ScienceDirect
- A dataset of Bangla YouTube comments for relevance/sentiment analysis (CineXDrama) is reported (~14k comments). arXiv

---

12) Common failure cases (so you can debug intelligently)

1. **Sarcasm / memes**: “nostalgia” used jokingly
2. **Past without longing**: “আগে এটা ছিল না”—complaint
3. **Quoted lyrics**: old song lyrics trigger nostalgia label incorrectly
4. **“Nostalgia” about a person/event** without emotion
5. **Mixed sentiment**: “পুরনো দিন ভালো ছিল কিন্তু…” (nostalgic + critique)

Best fix is *not fancy modeling*—it’s:

- better annotation guideline + more examples of edge cases
- adding video title/context
- script-aware handling

If you tell me (1) roughly how many comments you have, and (2) how much is **Bangla script vs romanized vs Banglish**, I can recommend the best starting model + preprocessing combo (BanglaBERT vs BanglishBERT vs MuRIL) and a minimal experiment plan that gets you to a solid F1 quickly.

Sources